- Since they're not available until runtime, you can't use them with the --example option to run a single spec.
- Your specs' output can be misleading if you change your setup code and forget to update the describe or context documentation.
- Specs written in a one-liner style carry extra cognitive load; you have to understand how subject and is_expected/should relate.

The one case when we do recommend one-liner syntax is when the generated description is a near-exact duplicate of what you would have written by hand. For a good example, see the specs for the Mustermann string-matching library.[2]

Your Turn

In this chapter, you've learned the major parts of an expectation: the expect method, the subject, to/not_to, and the matcher. You wrote some simple expectations using expect and a few of RSpec's built-in matchers. You saw how to combine matchers by passing them into one another, and by combining them with and/or.

In the next chapter, we'll take you on a tour of the matchers that RSpec ships with. But first, test your newfound knowledge of expectations with a quick exercise.

Exercise

We've prepared an unfinished spec file that will give you the chance to try out the concepts explained in this chapter. We encourage you to download it, run it, and edit it until you've gotten all the specs to pass consistently:

10-exploring-rspec-expectations/exercises/data_generator_spec.rb

```
require 'date'

class DataGenerator
  def boolean_value
    [true, false].sample
  end

  def email_address_value
    domain = %w[ gmail.com yahoo.com aol.com hotmail.com ].sample
    username_characters = (0..9).to_a + ('a'..'z').to_a + ('A'..'Z').to_a
    username_length = rand(5) + 3
    username = Array.new(username_length) { username_characters.sample }.join

    "#{username}@#{domain}"
  end
```

2. https://github.com/sinatra/mustermann/blob/v0.4.0/mustermann/spec/sinatra_spec.rb